

Тема 1. Введение в Node.js.



хекслет колледж



Цель занятия:

Сформировать у студентов базовое понимание того, что такое Node.js, где и зачем он применяется, а также получить первый практический опыт - установить Node.js и запустить простую программу.

Учебные вопросы:

1. Что такое Node.js и применяется.
2. Установка Node.js.
3. Первая программа.
4. Глобальные объекты Node.js.

1. Что такое Node.js и применяется.

История создания.

Node.js создан американским программистом Райаном Далем (Ryan Dahl) и впервые представлен в 2009 году на конференции JSConf EU.

До появления Node.js JavaScript работал исключительно в браузере.

Райан Даль поставил цель - вынести JavaScript за пределы браузера и дать возможность писать на нём серверный код.

Что такое Node.js?

Node.js - это среда выполнения JavaScript вне браузера, построенная на движке V8 (тот же движок, что используется в Google Chrome).

Простыми словами: Node.js позволяет запускать JavaScript-код на компьютере или сервере - как обычную программу, без браузера.

💡 Node.js - это не язык программирования и не фреймворк. Это runtime-среда (среда выполнения).

Движок V8

V8 - это высокопроизводительный движок с открытым исходным кодом, разработанный компанией Google. Он компилирует JavaScript напрямую в машинный код, что обеспечивает высокую скорость выполнения.

Node.js берёт V8 и расширяет его возможностями, которых нет в браузере:

- работа с файловой системой
- работа с сетью
- доступ к операционной системе

Однопоточность и асинхронность

Node.js работает в одном потоке, но при этом эффективно справляется с большим количеством задач одновременно за счёт событийного цикла (Event Loop).

Принцип работы:

- Получил запрос → не ждёт ответа → занялся следующим

Когда ответ готов → вернулся и обработал

Это называется неблокирующая (асинхронная) модель ввода-вывода.

Чем Node.js отличается от браузерного JavaScript?

	Браузер	Node.js
Где работает	В браузере	На компьютере / сервере
Глобальный объект	<code>window</code>	<code>global</code>
Доступ к файлам	✗	✓
DOM, BOM	✓	✗
Работа с сетью	Ограничена	Полная
Модули	ES Modules	CommonJS (<code>require</code>)

Где применяется Node.js?

Backend-серверы и REST API:

- Node.js широко используется для создания серверной части веб-приложений. На нём строят API, к которым обращаются мобильные приложения и браузеры.

CLI-утилиты:

- Многие инструменты разработчика написаны на Node.js и запускаются в терминале. Примеры: npm, webpack, eslint, prettier.

Работа с файловой системой:

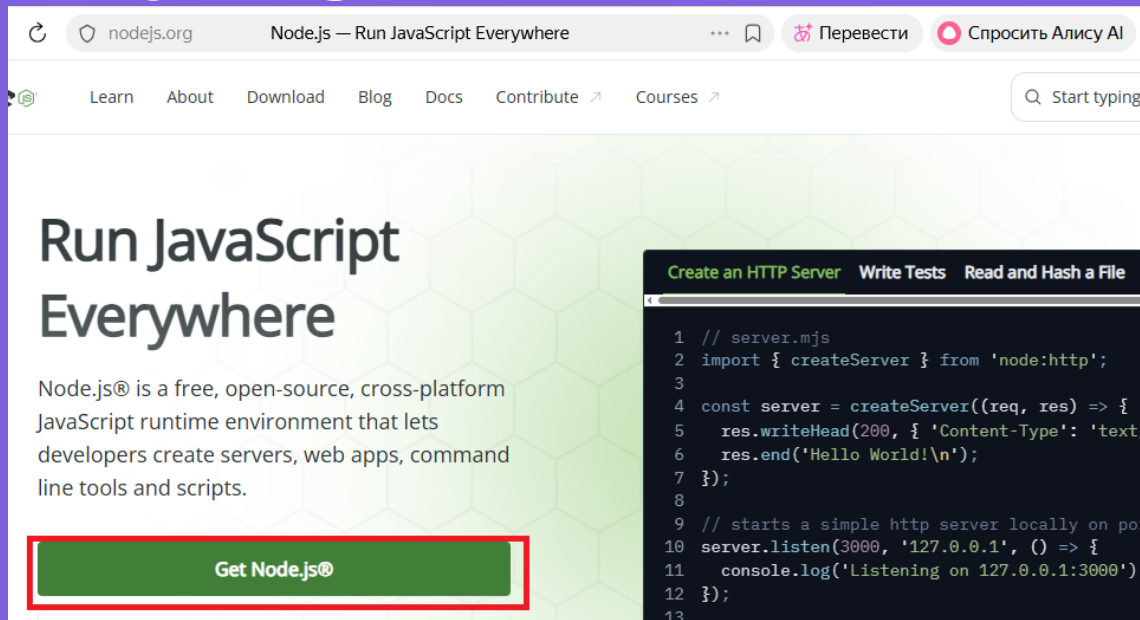
- Скрипты для автоматизации: переименование файлов, генерация отчётов, обработка данных.

Итог:

Что такое	Среда выполнения JavaScript вне браузера
На чём основан	Движок V8 от Google
Главная особенность	Асинхронная, неблокирующая модель
Где используется	Серверы, API, CLI-утилиты, автоматизация

2. Установка Node.js.

Скачать Node.js можно с официального сайта: **nodejs.org**



Для обучения и рабочих проектов всегда выбирайте LTS:

Download Node.js®

Get Node.js® v24.14.0 **(LTS)** for Windows using Docker with npm

Info Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:24-slim
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:24-slim
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.14.1".
12
13 # Verify npm version:
14 npm -v # Should print "11.11.0".
```

PowerShell </> Copy to clipboard

Docker is a containerization platform. If you encounter any issues please visit [Docker's website](#)

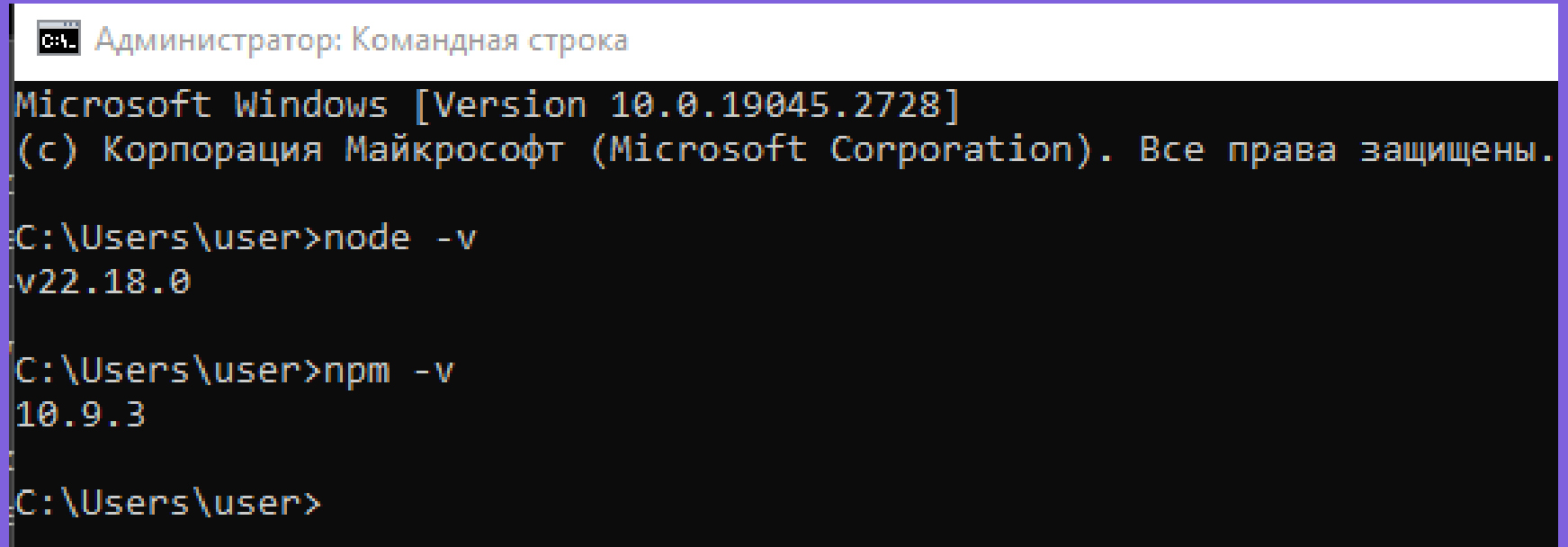
Or get a prebuilt Node.js® for Windows running a x64 architecture.

Windows Installer (.msi) Standalone Binary (.zip)

Установка на Windows:

- Перейти на nodejs.org
- Скачать установщик .msi (LTS версия)
- Запустить установщик → Next → Next → Install
- Галочку "Add to PATH" оставить включённой - это важно

После установки открыть командную строку или PowerShell и проверить:



```
Администратор: Командная строка
Microsoft Windows [Version 10.0.19045.2728]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\user>node -v
v22.18.0

C:\Users\user>npm -v
10.9.3

C:\Users\user>
```

Выбор редактора кода

Для работы с Node.js рекомендуется **Visual Studio Code**:

- Бесплатный
- Встроенный терминал
- Подсветка синтаксиса и автодополнение для JavaScript/Node.js
- Скачать: code.visualstudio.com

Итог:

1	Перейти на nodejs.org
2	Скачать LTS версию
3	Установить с галочкой "Add to PATH"
4	Проверить: <code>node -v</code> и <code>npm -v</code>
5	Открыть VS Code

3. Первая программа на Node.js.

Node.js можно использовать в двух режимах:

- REPL - интерактивный режим прямо в терминале
- Запуск файла - выполнение готового .js файла

REPL - интерактивный режим

REPL расшифровывается как Read – Eval – Print – Loop:

- Read - читает введенную команду
- Eval - выполняет её
- Print - выводит результат
- Loop - ожидает следующую команду

Запуск REPL - просто ввести в терминале:

node

Появится приглашение `>` - можно вводить JavaScript.

Выход: `.exit`

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
● PS C:\Users\user\Documents\nodeproject> node
Welcome to Node.js v22.18.0.
Type ".help" for more information.
> 2 + 2
4
> .exit
○ PS C:\Users\user\Documents\nodeproject> 
```

Первая программа - запуск файла

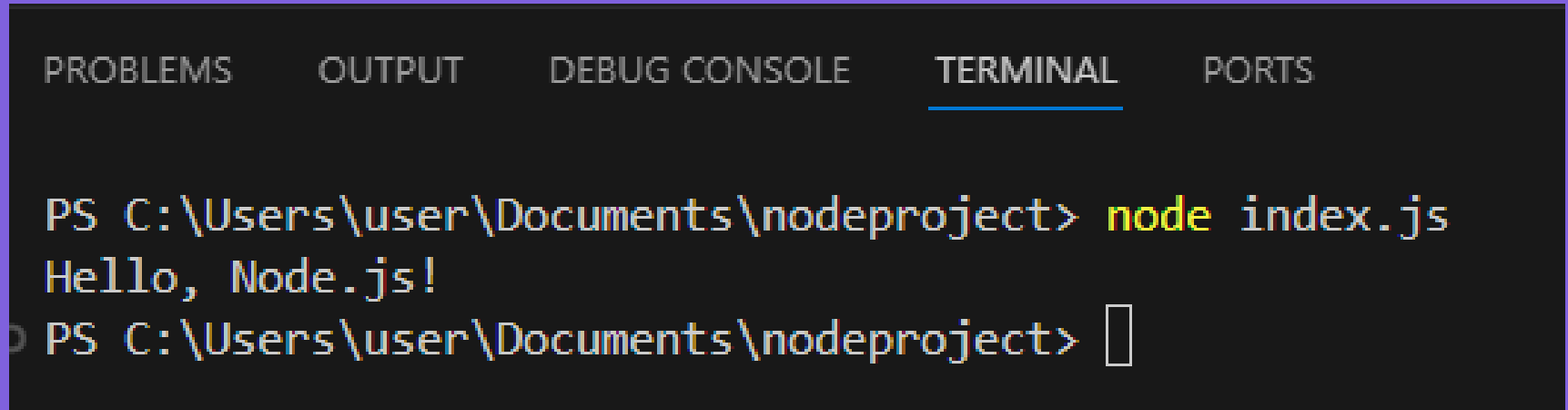
Создадим папку проекта, внутри файл index.js с кодом:

```
console.log('Hello, Node.js!');
```

В терминале выполним:

```
node index.js
```

Результат:



A screenshot of a Visual Studio Code terminal window. The terminal has a dark background with a light blue title bar. At the top, there are five tabs: 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined with a blue line), and 'PORTS'. Below the tabs, the terminal shows a PowerShell prompt 'PS C:\Users\user\Documents\nodeproject>' followed by the command 'node index.js' in yellow. The output 'Hello, Node.js!' is displayed in light blue. Below that, the prompt is shown again with a white cursor character.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\user\Documents\nodeproject> node index.js
Hello, Node.js!
PS C:\Users\user\Documents\nodeproject> █
```

Отличие от браузерного JavaScript:

В браузере JavaScript взаимодействует с DOM, имеет доступ к window, document, alert и т.д.

В Node.js всего этого нет - здесь нет браузера, нет страницы.

Зато есть доступ к файловой системе, сети и операционной системе.

Быстрая проверка в index.js:

```
PS C:\Users\user\Documents\nodeproject> node
Welcome to Node.js v22.18.0.
Type ".help" for more information.
> console.log(typeof window);
undefined
undefined
> console.log(typeof document);
undefined
undefined
> console.log(typeof process);
object
undefined
> 
```

Итог:

REPL	Интерактивный режим, запуск командой <code>node</code>
Запуск файла	<code>node index.js</code>
Отличие от браузера	Нет <code>window</code> , <code>document</code> , <code>DOM</code>

4. Глобальные объекты Node.js.

В браузере главный глобальный объект - `window`. Через него доступны все встроенные функции: `window.alert()`, `window.console`, `window.setTimeout()` и т.д.

В Node.js роль главного глобального объекта играет `global`. Всё, что находится в `global`, доступно в любом месте программы без импорта.

Объект `global`

`global` - это главный глобальный объект Node.js, в котором хранятся все встроенные функции и свойства, доступные в любом месте программы без импорта.

Это «общая комната» программы - всё, что лежит в `global`, доступно из любого файла в любой момент.

Если в браузере такой комнатой был `window`, то в Node.js его роль играет `global`.

```
> console.log(global) // все глобальные свойства и методы
<ref *1> Object [global] {
  global: [Circular *1],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function: setImmediate] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function: setTimeout] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  queueMicrotask: [Function: queueMicrotask].
```

Всё что находится в `global` - доступно в любом месте программы без импорта:

```
setTimeout(() => {  
  console.log('Привет!');  
}, 1000);
```

Мы не импортируем `setTimeout` - он находится в `global`. Полная запись выглядела бы так:

```
global.setTimeout(() => {  
  console.log('Привет!');  
}, 1000);
```

Результат одинаковый - просто `global`. можно не писать.

Объект process

Определение: `process` - это глобальный объект Node.js, который содержит информацию о текущем запущенном процессе и предоставляет методы для взаимодействия с ним.

`process` - это объект, который содержит информацию о запущенной программе. Через него можно узнать версию Node.js, на какой системе работает программа, и какие аргументы ей передали при запуске.

Аргументы командной строки -
`process.argv`

Когда мы запускаем программу, можно передать ей дополнительные данные прямо в терминале.

Пример:

```
index.js
```

```
1 console.log(process.argv);  
2
```

```
PS C:\Users\user\Documents\nodeproject> node index.js Alex  
[  
  'C:\\Program Files\\nodejs\\node.exe',  
  'C:\\Users\\user\\Documents\\nodeproject\\index.js',  
  'Alex'  
]
```

Версия и платформа

```
console.log(process.version);  
console.log(process.platform);
```

```
PS C:\Users\user\Documents\nodeproject> node index.js  
v22.18.0  
win32
```

Завершение программы - `process.exit()`

```
console.log('Начало');  
process.exit();  
console.log('Это не выведется'); // до сюда программа не дойдёт
```

Выводы:

- Node.js - среда выполнения JavaScript вне браузера
- Node.js - это не язык и не фреймворк, это runtime-среда
- Для обучения и работы всегда выбирать версию LTS
- Проверка установки: `node -v` и `npm -v`
- REPL - интерактивный режим для быстрой проверки кода, запуск командой `node`
- Запуск программы из файла: `node index.js`
- В Node.js нет `window` и `DOM` - это серверная среда, не браузер
- Главный глобальный объект - `global`, аналог браузерного `window`
- `process` - объект с информацией о запущенной программе
- `process.argv` - массив аргументов командной строки, данные начинаются с индекса [2]

Контрольные вопросы

1. Что такое Node.js и на чём он основан?
2. Чем Node.js отличается от JavaScript в браузере? Назовите минимум два отличия.
3. Что такое LTS-версия и почему её рекомендуют для обучения?
4. Как проверить, что Node.js установлен корректно? Какие команды нужно выполнить?
5. Что такое REPL? Как его запустить и как выйти из него?
6. Как запустить файл index.js через терминал?
7. Что такое объект global? Чем он похож на window в браузере?
8. Что хранится в process.argv? С какого индекса начинаются пользовательские аргументы и почему?
10. Что произойдёт, если вызвать process.exit() в середине программы